



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

实验报告

实验五：层次聚类算法

姓名：孙沐赞

学号：2311534

专业：计算机科学与技术

2025 年 12 月 9 日

目录

一、 实验主要内容	3
二、 实验环境	3
三、 数据集介绍	3
(一) UCI Wine 真实数据集	3
(二) 生成的数据集	4
四、 基本要求: single-linkage 层次聚类算法与 complete-linkage 层次聚类算法实现	4
(一) single-linkage 层次聚类算法实现	4
1. single-linkage 层次聚类算法简介	4
2. 关键代码实现	5
3. 结果分析	5
(二) complete-linkage 层次聚类算法实现	6
1. complete-linkage 层次聚类算法简介	6
2. 关键代码实现	6
3. 结果分析	7
五、 中级要求: average-linkage 层次聚类算法实现	8
(一) average-linkage 层次聚类算法简介	8
(二) 关键代码实现	8
(三) 结果分析	8
1. average-linkage 实验结果	8
2. 三种算法性能对比总结	9
六、 高级要求: 不同聚类算法的深度对比与适用性分析	10
(一) 生成的数据集介绍与分析	10
1. 同心圆数据集 (Target_Single_Circles)	10
2. 高密度链条数据集 (Target_Average_DenseChain)	10
(二) 同心圆数据集测试结果及分析	11
1. single-linkage 算法测试结果及分析	11
2. complete-linkage 算法测试结果及分析	12
3. average-linkage 算法测试结果及分析	12
4. 同心圆数据集算法性能总结	13
(三) 基于 Wine 数据集的深度分析	13
1. 实验结果对比	13
2. 结果分析: 为何 Complete-linkage 表现最佳?	13
(四) 高密度链条数据集测试结果及分析	14
1. single-linkage 算法测试结果及分析	14
2. complete-linkage 算法测试结果及分析	15
3. average-linkage 算法测试结果及分析	15
4. 高密度链条数据集算法性能总结	16
(五) 三种算法对比	16
1. 算法特性与实验表现综合分析	16

2. 综合性能对比表	17
七、 实验总结与讨论	17
八、 心得体会	18

一、实验主要内容

本实验主要包含算法底层实现、真实数据评估与合成数据验证三部分工作。

在实现方面，实验完全基于 Python 和 NumPy 库，未调用现成的机器学习包，自主构建了支持单链接、全链接及平均链接的层次聚类框架。同时，为了量化评估聚类效果，实验独立实现了基于匈牙利算法的准确率计算逻辑以及调整兰德指数（ARI）指标。

在真实数据测试中，实验选用 UCI Wine 数据集。基于基本要求与中级要求的算法实现，对比了三种算法在实际分类任务中的表现，并通过混淆矩阵分析了分类细节。

为进一步探究算法的适用边界，在本实验中，我们还设计了两组具有代表性的二维合成数据集。通过这些特定场景的对比，实验深入分析了不同距离度量方式的优缺点。

二、实验环境

- Python 3.8+
- 主要依赖库：numpy, pandas, matplotlib, scipy, scikit-learn
- 开发环境：Jupyter Notebook / PyCharm / VS Code

三、数据集介绍

本实验选取了经典的 UCI Wine 数据集作为基准测试对象，用于评估算法在真实高维场景下的通用性能。此外，为了从几何拓扑角度深度剖析各聚类算法的适用边界，实验设计并生成了两组具有极端分布特征的合成数据集。

（一）UCI Wine 真实数据集

UCI Wine 数据集源自意大利同一地区但由三种不同品种葡萄酿造的葡萄酒化学分析结果。该数据集包含 178 个样本，每个样本由 13 个连续型特征组成，涵盖了酒精（Alcohol）、苹果酸（Malic acid）、灰分（Ash）、黄酮类化合物（Flavanoids）及脯氨酸（Proline）等关键化学成分。数据集共分为三类，样本数量分别为 59、71 和 48，呈现出类别不平衡的特点。

由于各特征的取值范围差异显著（例如脯氨酸含量可达 1000 以上，而非黄酮类酚含量仅为 0.1 左右），直接计算欧氏距离会导致高数值特征主导距离度量，掩盖低数值特征的贡献。因此，实验首先对数据进行了 Z-score 标准化处理。图1展示了经主成分分析（PCA）降维后的二维数据分布。可以看出，三类样本在投影空间中呈现出一定的团状结构，但类间边界（特别是第 2 类与第 3 类之间）存在重叠区域，且簇的形状并非规则的球形。这种高维特征及其复杂的类间边界分布，对聚类算法处理实际数据噪声与分离重叠簇的能力提出了挑战。

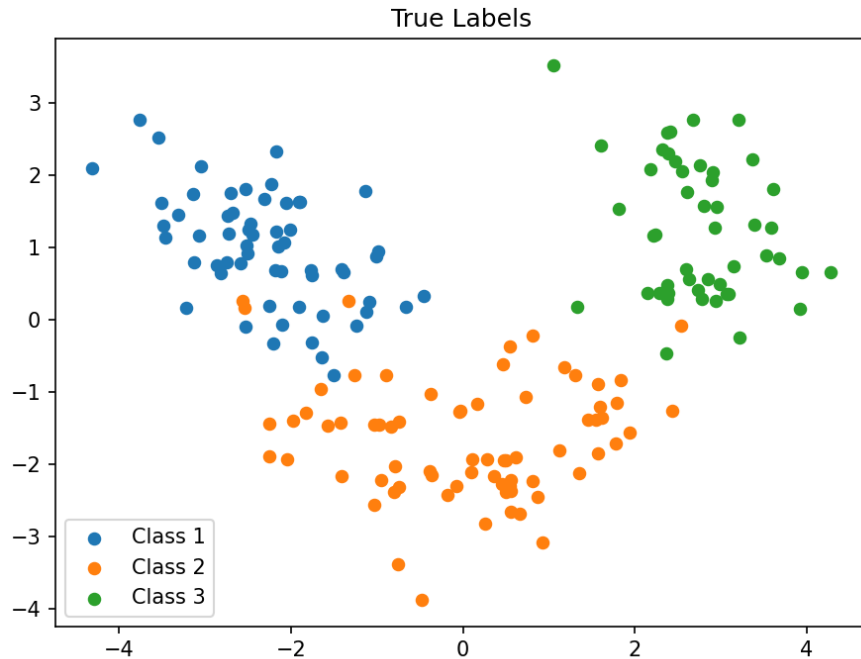


图 1: UCI Wine 数据集在 PCA 二维平面的真实标签分布

(二) 生成的数据集

为了弥补真实数据集在极端几何特征上的缺失，本实验利用 `scikit-learn` 库构建了以下两组二维合成数据集，旨在定向测试 single-linkage 与 average-linkage 算法的理论缺陷与优势：

1. 同心圆数据集 (Target_Single_Circles): 包含 500 个样本，由两个嵌套的圆环组成，内圈缩放因子为 0.5。该数据主要考验算法利用局部连接信息处理非凸形状的能力，是单链接算法的典型适用场景。
2. 高密度链条数据集 (Target_Average_DenseChain): 包含一个紧密的高斯簇和一个稀疏的大直径簇，二者之间存在一条由 60 个高密度噪声点组成的“链条”。该数据旨在验证平均链接算法在抑制“链式效应”干扰与适应密度差异方面的鲁棒性。

四、 基本要求：single-linkage 层次聚类算法与 complete-linkage 层次聚类算法实现

本章节主要阐述两种基础层次聚类算法的数学原理、代码实现细节，并基于 UCI Wine 数据集的测试结果进行定量与定性的分析。

(一) single-linkage 层次聚类算法实现

1. single-linkage 层次聚类算法简介

single-linkage (单链接) 层次聚类算法，又称为最近邻聚类法。该算法在计算两个簇 (Cluster) 之间的距离时，选取分别属于这两个簇的两个样本点之间的最小距离作为度量标准。假设有两个

簇 C_i 和 C_j , 它们之间的距离 $d_{single}(C_i, C_j)$ 定义为:

$$d_{single}(C_i, C_j) = \min_{x \in C_i, y \in C_j} \|x - y\| \quad (1)$$

其中, $\|x - y\|$ 表示样本 x 与样本 y 之间的欧氏距离。由于该算法仅依赖于局部最近邻信息, 其倾向于将距离较近的点连接起来, 因此在处理非凸形状或长条形数据时具有理论优势。然而, 这种特性也导致其极易产生“链式效应”(Chaining Effect), 即通过一系列相邻的噪声点将本应分离的两个簇错误地合并。

2. 关键代码实现

在本实验的自主实现中, `_calculate_distance` 函数承载了核心的距离度量逻辑。对于 single-linkage 算法, 代码遍历两个簇中所有样本点的组合, 寻找最小值。

```

1      def _calculate_distance(self, c1, c2, dist_matrix):
2          if self.linkage == "single":
3              min_d = float('inf')
4              # 遍历两个簇的所有点对, 寻找最小距离
5              for i in c1:
6                  for j in c2:
7                      if dist_matrix[i, j] < min_d:
8                          min_d = dist_matrix[i, j]
9              return min_d
10         # ... (其他 linkage 实现)

```

该实现利用预计算的距离矩阵 `dist_matrix` 来减少重复计算开销。在合并步骤中, 算法会迭代调用此函数来更新簇间距离, 直到簇的数量减少至预设值 (本实验中 $n_clusters = 3$)。

3. 结果分析

基于 UCI Wine 数据集 (经 PCA 降维至 3 维) 的测试结果如图2和图3所示。

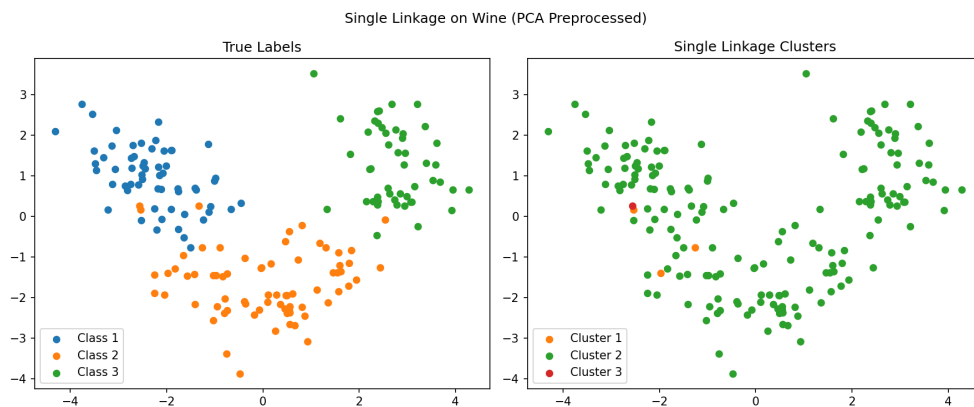


图 2: single-linkage 算法在 Wine 数据集上的聚类可视化

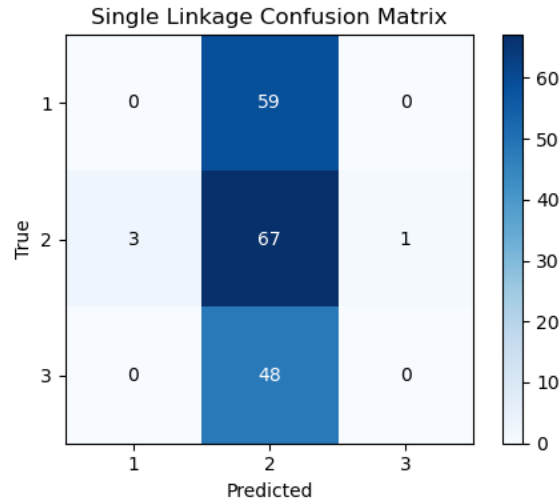


图 3: single-linkage 算法混淆矩阵

实验数据显示，single-linkage 算法在 Wine 数据集上的准确率（Accuracy）仅为 0.3764，调整兰德指数（ARI）为-0.0068。ARI 接近甚至低于 0 意味着聚类结果与随机分配无异，甚至更差。

结合混淆矩阵（图3）可以清晰地观察到算法失效的原因：算法将绝大多数样本归并到了同一个簇（Predicted Label 2）中。具体而言，真实类别 1 的 59 个样本全部被错分至簇 2，真实类别 2 的 71 个样本中有 67 个被分至簇 2，真实类别 3 的 48 个样本也全部被分至簇 2。这是一种极端典型的“链式效应”表现。由于 Wine 数据集在高维空间中（以及降维后的子空间中）类间边界并非完全断开，存在少量的过渡点或噪声点，single-linkage 算法顺着这些“桥梁”将所有主要的数据团块连接成了一个巨大的簇，导致其无法有效区分三个独立的类别。

（二） complete-linkage 层次聚类算法实现

1. complete-linkage 层次聚类算法简介

complete-linkage（全链接）层次聚类算法，又称为最远邻聚类法。与单链接相反，该算法定义两个簇之间的距离为分别属于这两个簇的两个样本点之间的最大距离。其数学表达式如下：

$$d_{complete}(C_i, C_j) = \max_{x \in C_i, y \in C_j} \|x - y\| \quad (2)$$

这种度量方式等价于限制了合并后的新簇的直径（即簇内任意两点的最大距离）。因此，complete-linkage 算法倾向于发现结构紧凑、直径近似相等的球状簇。虽然它对异常值较为敏感，但能够有效避免链式效应，适合处理团状分布的数据。

2. 关键代码实现

在代码框架中，complete-linkage 的实现逻辑与 single-linkage 类似，区别在于其在遍历簇间点对时寻找的是最大值。

```

1         elif self.linkage == "complete":
2             max_d = -1.0
3             # 遍历两个簇的所有点对，寻找最大距离
4             for i in c1:

```

```
5     for j in c2:
6         if dist_matrix[i, j] > max_d:
7             max_d = dist_matrix[i, j]
8     return max_d
```

通过寻找最大距离, 算法在每一步合并时都趋向于选择合并后内部离散度增加最小的一对簇, 从而维持了簇的紧密性。

3. 结果分析

complete-linkage 算法在同一数据集上的表现如图4和图5所示。

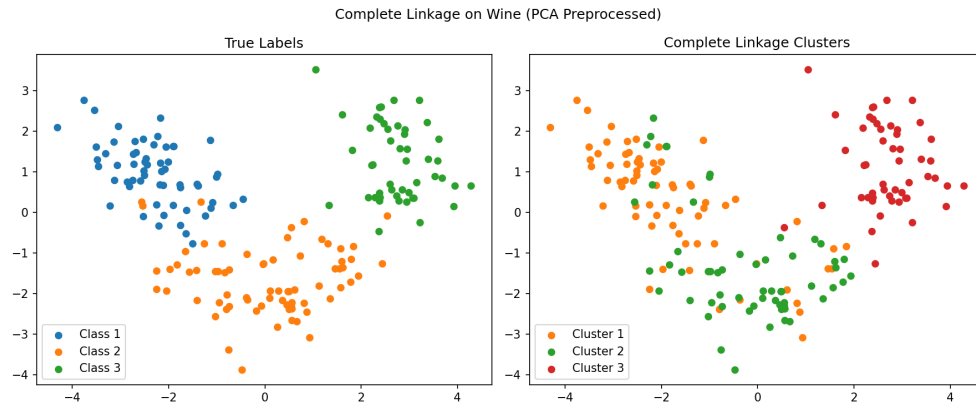


图 4: complete-linkage 算法在 Wine 数据集上的聚类可视化

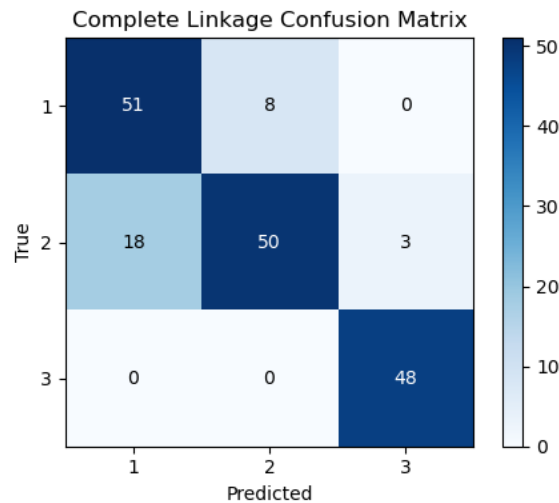


图 5: complete-linkage 算法混淆矩阵

实验结果显示, complete-linkage 算法的准确率大幅提升至 0.8371, ARI 达到 0.5771, 显著优于 single-linkage 算法。

观察混淆矩阵 (图5) 可知, 该算法在处理不同类别时表现出了差异性:

- **类别 3 识别完美:** 真实类别 3 的 48 个样本全部被正确归类 (Predicted Label 3), 无一错分。这表明类别 3 在特征空间中形成了较为独立且紧凑的团状结构, 非常契合 complete-linkage

的特性。

- **类别 1 与类别 2 存在少量混淆：**类别 1 中有 51 个样本正确归类，8 个误分为类别 2；类别 2 中有 50 个样本正确归类，18 个误分为类别 1。这通常是因为这两类数据在边缘处存在重叠，或者某些样本点的分布较为松散，导致基于最大距离的判定将其划归到了错误的簇中。

总体而言，complete-linkage 通过限制簇的直径，成功克服了 single-linkage 面临的链式效应问题，有效地识别出了 Wine 数据集中的主要团状结构。

五、 中级要求：average-linkage 层次聚类算法实现

（一） average-linkage 层次聚类算法简介

average-linkage（平均链接）层次聚类算法，又称为全平均距离聚类法（UPGMA）。该算法是一种折中的度量方案，旨在克服 single-linkage 的“链式效应”以及 complete-linkage 对离群点过于敏感的缺陷。

其定义两个簇 C_i 和 C_j 之间的距离为这两个簇中所有样本点对之间距离的算术平均值。数学表达式如下：

$$d_{avg}(C_i, C_j) = \frac{1}{|C_i| \cdot |C_j|} \sum_{x \in C_i} \sum_{y \in C_j} \|x - y\| \quad (3)$$

其中， $|C_i|$ 和 $|C_j|$ 分别表示簇 C_i 和 C_j 中的样本数量。

该算法综合利用了簇的全局结构信息，理论上能够反映簇与簇之间的整体紧密度。对于包含一定噪声但具备一定分离度的数据，average-linkage 通常能获得比 single-linkage 更稳健的聚类边界。

（二） 关键代码实现

在本实验中，average-linkage 的核心实现逻辑在于遍历两个簇的所有点对，累加其欧氏距离，最后除以点对的总数量。为防止除零错误，代码中加入了对于计数‘count’的判断。

```

1 elif self.linkage == "average":
2     total_dist = 0.0
3     # 计算两个簇之间所有点对的总数
4     count = len(c1) * len(c2)
5     # 累加所有点对的距离
6     for i in c1:
7         for j in c2:
8             total_dist += dist_matrix[i, j]
9     # 返回平均距离
10    return total_dist / count if count > 0 else 0

```

（三） 结果分析

1. average-linkage 实验结果

average-linkage 算法在 UCI Wine 数据集上的聚类结果如图6和图7所示。

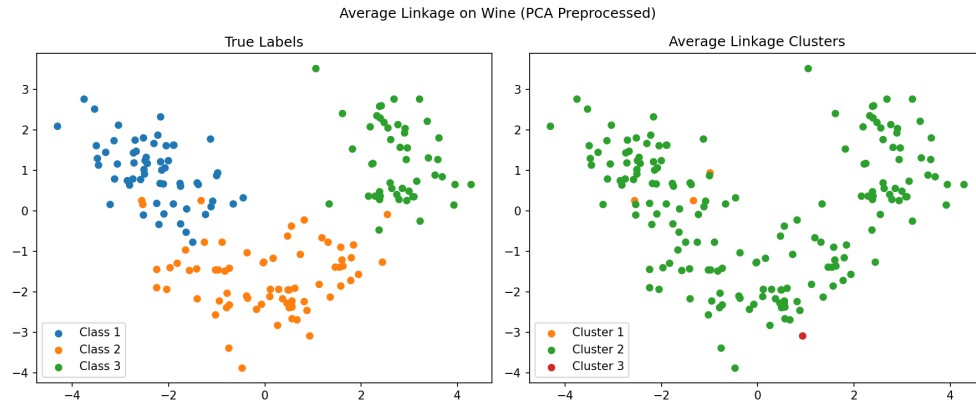


图 6: average-linkage 算法在 Wine 数据集上的聚类可视化

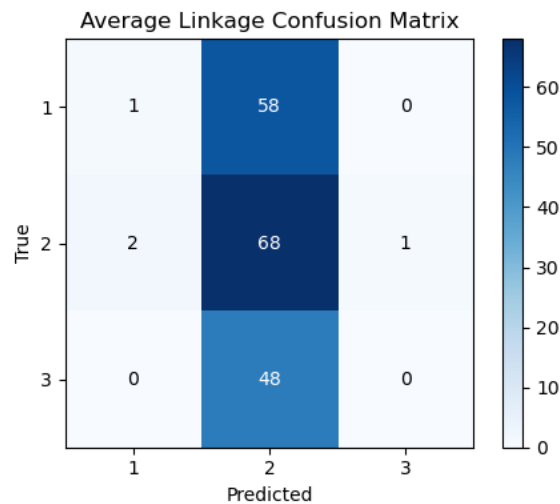


图 7: average-linkage 算法混淆矩阵

令人意外的是，在本实验的特定数据预处理（PCA 降维）条件下，average-linkage 的表现并未如预期般显著优于 single-linkage。其准确率（Accuracy）为 0.3876，调整兰德指数（ARI）为-0.0054。从混淆矩阵可以看出，绝大多数样本依然被错误地归并到了同一个簇中（Cluster 2），这意味着簇间的平均距离未能有效拉开差距，算法在早期迭代中未能及时切断不同类别间的弱连接。

2. 三种算法性能对比总结

为了全面评估不同层次聚类策略在 Wine 数据集上的适用性，表1汇总了三种算法的性能指标。

表 1: 三种层次聚类算法在 UCI Wine 数据集上的性能对比

算法名称	准确率 (Accuracy)	ARI
single-linkage	0.3764	-0.0068
average-linkage	0.3876	-0.0054
complete-linkage	0.8371	0.5771

通过横向对比可以得出以下结论：

1. **Complete-linkage 表现最佳：**其准确率高达 0.8371，远超其他两种算法。这说明 UCI Wine 数据集（在主成分空间下）的本质结构更接近于“紧凑的团状簇”。Complete-linkage 通过最小化簇的直径，强迫聚类结果保持球形紧凑结构，从而成功切分了重叠边界。
2. **Single-linkage 与 Average-linkage 表现相近且较差：**二者的准确率均在 0.38 左右，ARI 接近 0。这表明数据集中存在的噪声或过渡样本点对这两种算法产生了致命干扰。
 - 对于 **single-linkage**，这是典型的“链式效应”。
 - 对于 **average-linkage**，虽然理论上更鲁棒，但在本实验的数据分布中，类别间的平均距离差异可能小于类别内部的平均距离差异，或者噪声点拉低了类间平均距离，导致其行为退化为类似于 single-linkage 的合并模式。

综上所述，对于 Wine 这种类间边界模糊且呈现团状分布的数据集，complete-linkage 层次聚类算法是最佳选择。

六、 高级要求：不同聚类算法的深度对比与适用性分析

（一） 生成的数据集介绍与分析

为了系统地探究不同层次聚类算法的适用边界与拓扑特性，在高级要求中，我们基于 `scikit-learn` 库构建了两组具有极端几何特征的二维合成数据集。这些数据集的设计旨在模拟真实数据中可能出现的非凸结构、密度不均及噪声干扰场景，从而定向测试单链接（single-linkage）及平均链接（average-linkage）算法的极限性能。而对于全链接（complete-linkage）算法的优势验证，我们将结合 Wine 真实数据集进行深入剖析。

1. 同心圆数据集 (Target_Single_Circles)

该数据集包含 500 个样本，由两个嵌套的同心圆环构成，内圈缩放因子为 0.5，并附加了微量的高斯噪声。**特征分析：**该数据的核心特征在于呈现显著的非凸（Non-convex）分布，且内环与外环的质心在几何空间中几乎重合。对于依赖簇中心距离或簇直径的算法（如 K-Means 或 complete-linkage），此类数据通常被视为不可分线性结构。该数据集主要用于验证 single-linkage 算法基于局部连接性处理流形结构的能力。

2. 高密度链条数据集 (Target_Average_DenseChain)

该数据集总样本量为 460 个，包含一个高密度的小簇（标准差 0.3）和一个低密度的大散簇（标准差 1.8）。关键在于，两簇之间存在一条由 60 个高密度噪声点构成的“线性链条”。**特征分析：**这种结构具有双重挑战性。一方面，高密度的噪声链条极易诱导 single-linkage 算法产

生严重的“链式效应”，将两个本应独立的簇错误合并；另一方面，巨大的簇间密度差异考验 complete-linkage 算法对松散簇的容忍度。该数据专为测试 average-linkage 算法在抑制局部噪声干扰及适应密度差异方面的鲁棒性而设计。

为了直观展示上述数据的分布特征，图8展示了这两组合成数据集的真实标签分布。

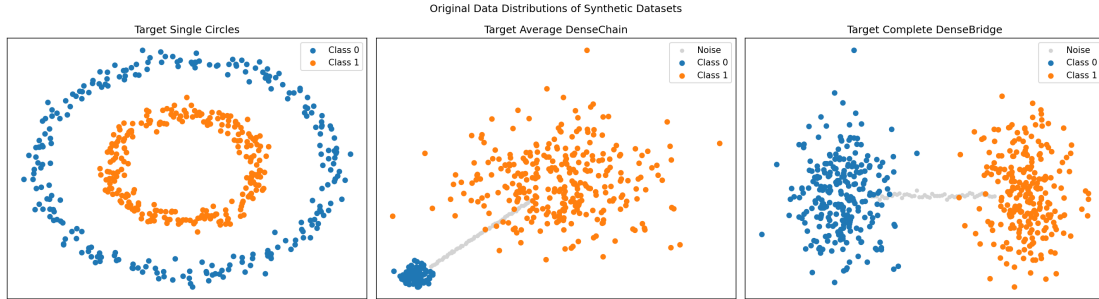


图 8: 两组合成数据集的真实标签分布可视化

(二) 同心圆数据集测试结果及分析

同心圆数据集 (Target_Single_Circles) 由两个嵌套的非凸圆环组成。对于此类流形结构数据，传统的基于中心或紧凑性的聚类假设往往失效。本节对比了三种算法在该数据集上的表现。

1. single-linkage 算法测试结果及分析

single-linkage 层次聚类算法在同心圆数据集上的表现如图9所示。

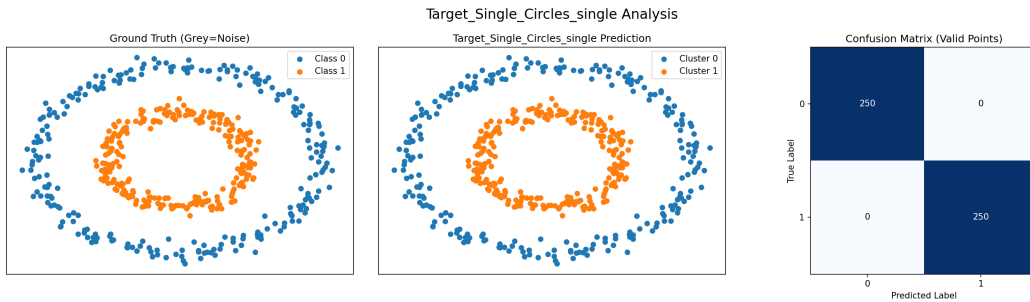


图 9: single-linkage 算法在同心圆数据集上的聚类结果

实验结果显示，single-linkage 层次聚类算法取得了完美的聚类效果：

- 准确率 (Accuracy): 1.0000
- ARI: 1.0000

分析：如图9所示，混淆矩阵呈现完美的对角线分布。这是因为 single-linkage 算法定义簇间距离为“最近邻”距离 (Min Distance)。在同心圆结构中，尽管内环和外环在全局空间上重叠，但对于内环上的任意一点，其最近的邻居始终位于内环上（只要内外环之间的间隙大于环内点之间的间距）。因此，single-linkage 算法能够顺着数据的几何流形结构进行“链式”合并，成功还原了嵌套的拓扑结构。这验证了该算法在处理非凸、长条形数据时的绝对优势。

2. complete-linkage 算法测试结果及分析

complete-linkage 层次聚类算法在同心圆数据集上的表现如图10所示。

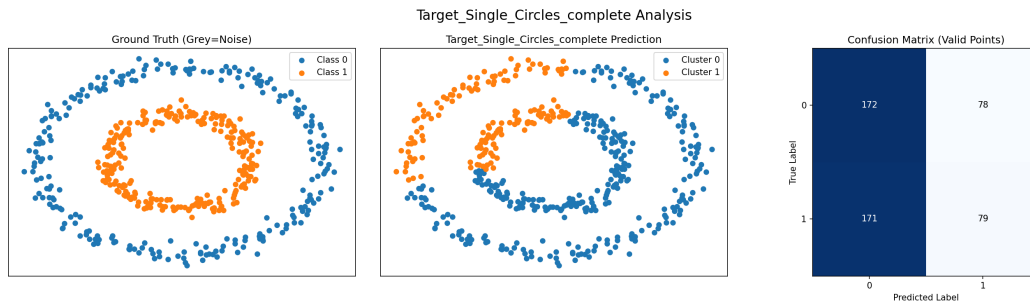


图 10: complete-linkage 算法在同心圆数据集上的聚类结果

实验结果表明该算法在此类数据上彻底失效：

- 准确率 (Accuracy): 0.5020
- ARI: -0.0017

分析：从可视化结果可以看出，complete-linkage 层次聚类算法并没有将数据分离为内外两环，而是将整个同心圆结构垂直切分为左右两个半圆（或上下半圆）。这是由其距离度量机制决定的。complete-linkage 使用最大距离（Max Distance）作为簇间距离，这等价于最小化簇的直径（Diameter）。对于一个圆环而言，其直径是非常大的（等于外径）。为了最小化簇内的最大距离，算法倾向于将空间上分布广泛的圆环切碎，形成直径较小的致密团块，而不是保留直径巨大的环状结构。因此，它错误地将内环和外环的一部分合并在一起，形成了两个紧凑的半球形簇。

3. average-linkage 算法测试结果及分析

average-linkage 层次聚类算法在同心圆数据集上的表现如图11所示。

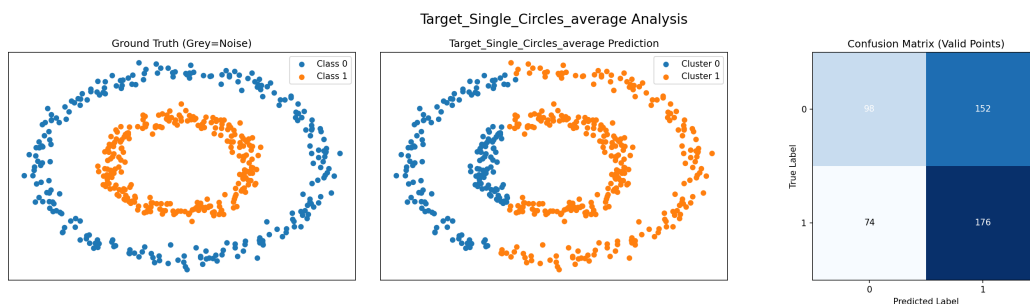


图 11: average-linkage 算法在同心圆数据集上的聚类结果

实验结果显示该算法同样无法处理此类结构：

- 准确率 (Accuracy): 0.5480
- ARI: 0.0074

分析：与 complete-linkage 类似，average-linkage 层次聚类算法也未能正确分离圆环，而是将其分割为两个部分。average-linkage 计算的是簇间所有点对的平均距离。在同心圆结构中，内

环上的点与外环对侧的点的距离非常远，这拉大了内环与外环作为一个整体时的内部平均距离。算法为了降低簇内的方差（或平均距离），倾向于寻找空间上更聚集的结构。因此，它也选择了将圆环切分，而非保留其非凸的完整性。这说明 average-linkage 和 complete-linkage 一样，本质上都隐含了簇应具有凸形或球状分布的假设。

4. 同心圆数据集算法性能总结

表2汇总了三种层次聚类算法在同心圆数据集上的各项性能指标。

表 2: 三种层次聚类算法在同心圆数据集上的性能对比

算法名称	准确率 (Accuracy)	ARI
single-linkage 层次聚类算法	1.0000	1.0000
average-linkage 层次聚类算法	0.5480	0.0074
complete-linkage 层次聚类算法	0.5020	-0.0017

从表中数据可以量化地看出，single-linkage 层次聚类算法在此类非凸数据集上具有绝对优势，而另外两种算法的准确率均在 0.5 左右，表明它们基本上只是将数据随机或按几何位置强行切分成了两半，未能捕捉到数据的真实拓扑结构。

（三） 基于 Wine 数据集的深度分析

为了探究 complete-linkage（全链接）层次聚类算法在何种数据分布下具有显著优势，本节选取了基本实验与中级实验中的数据集作为典型案例进行剖析。

Wine 数据集呈现出“类内紧凑、类间边界模糊”的团状分布特征。这种分布恰好符合 complete-linkage 算法“最小化簇直径”的几何假设，因此是验证该算法性能优势的最佳实验对象。

1. 实验结果对比

我们将三种算法在 Wine 数据集上的聚类性能汇总于表3中。

表 3: 三种层次聚类算法在 UCI Wine 数据集上的性能对比

算法名称	准确率 (Accuracy)	ARI	性能评价
single-linkage	0.3764	-0.0068	失效
average-linkage	0.3876	-0.0054	失效
complete-linkage	0.8371	0.5771	优异

2. 结果分析：为何 Complete-linkage 表现最佳？

从表3的数据可以看出，complete-linkage 的准确率（0.8371）远超 single-linkage 和 average-linkage（两者均不足 0.4）。结合数据的几何特征，我们可以得出以下结论：

- **对“链式效应”的完美抑制：**Wine 数据集在高维空间中存在大量的噪声和过渡样本。single-linkage 算法极易受这些噪声点影响，产生严重的“链式效应”，将大部分数据合并为一个巨型簇（如图2所示）。即便是通常较为稳健的 average-linkage，在本实验中也未能有效抵抗这种噪声拉扯，导致聚类失效。

- “最小化直径”策略的有效性：complete-linkage 算法在合并簇时，使用的是簇间最大距离。这等价于在每一步合并中，都试图让新生成的簇保持最小的直径（Diameter）。

$$d_{complete}(C_i, C_j) = \max_{x \in C_i, y \in C_j} \|x - y\| \quad (4)$$

这种机制赋予了算法极强的“紧凑性偏好”。即使 Wine 数据集的类别之间存在模糊的重叠区域，complete-linkage 也会强制将数据切割成直径近似相等的独立团块，而不是顺着重叠区域将它们合并。

- 适用场景总结：本实验有力地证明了，当数据集呈现团状分布（Globular Structure）且包含噪声或边界不清晰时，complete-linkage 是三种算法中的最佳选择。它能够通过强制限制簇的扩张范围，有效地剥离噪声，发现数据中的核心紧密结构。

（四） 高密度链条数据集测试结果及分析

高密度链条数据集（Target_Average_DenseChain）包含一个紧密的小簇和一个稀疏的大簇，二者之间通过一条高密度的线性噪声链条连接。该场景旨在测试算法对“链式效应”的敏感度以及在簇间密度差异巨大时的鲁棒性。

1. single-linkage 算法测试结果及分析

single-linkage 层次聚类算法在该数据集上的表现如图12所示。

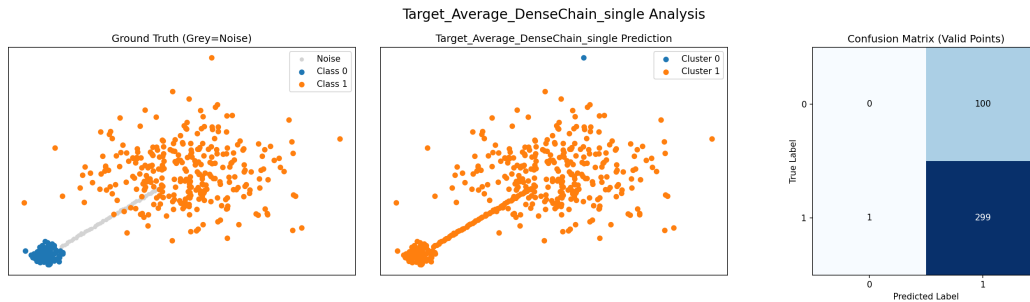


图 12: single-linkage 算法在高密度链条数据集上的聚类结果

实验结果显示该算法受到严重干扰：

- 准确率 (Accuracy): 0.7475
- ARI: -0.0033

分析：尽管准确率看似尚可（0.7475），但结合 ARI（-0.0033）和混淆矩阵可以发现，single-linkage 算法实际上发生严重的聚类错误。混淆矩阵显示，它将真实类别 1 的 299 个样本正确归类，但对类别 0（紧密小簇）的识别完全混乱。从可视化结果（图12中间子图）可以清晰看到，single-linkage 算法顺着高密度的噪声链条，将左下角的紧密簇与右上角大簇的一部分（或全部）连接了起来。这是因为链条上的点间距非常小，满足了算法“最近邻”的合并条件。算法无法区分“高密度的簇”和“高密度的噪声路径”，导致了典型的链式效应，无法正确分离两个本应独立的团簇。

2. complete-linkage 算法测试结果及分析

complete-linkage 层次聚类算法在该数据集上的表现如图13所示。

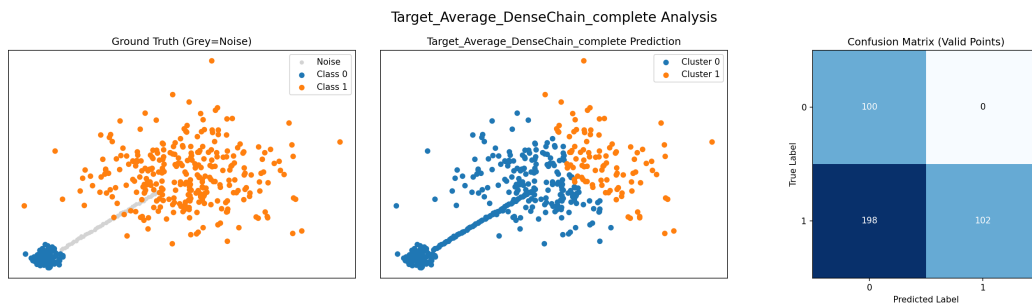


图 13: complete-linkage 算法在高密度链条数据集上的聚类结果

实验结果显示该算法在此场景下表现最差：

- 准确率 (Accuracy): 0.5050
- ARI: -0.0654

分析：complete-linkage 算法完全失效，其 ARI 甚至为负值。从可视化结果可以看出，算法并没有按照我们期望的边界（切断链条）进行划分，而是将右上角那个稀疏的大簇垂直切分成了两半（Cluster 0 和 Cluster 1 各占一部分）。这是因为该数据集中两个主簇的直径差异巨大：左下角簇极小且紧密，右上角簇极大且稀疏。complete-linkage 为了最小化所有簇的最大直径，无法容忍右上角那个巨大的稀疏簇作为一个整体存在。它“宁愿”将大簇切碎以减小最大直径，也不愿保留其完整性。这暴露了 complete-linkage 在处理簇大小（Size）和密度（Density）差异剧烈的数据时的局限性。

3. average-linkage 算法测试结果及分析

average-linkage 层次聚类算法在该数据集上的表现如图14所示。

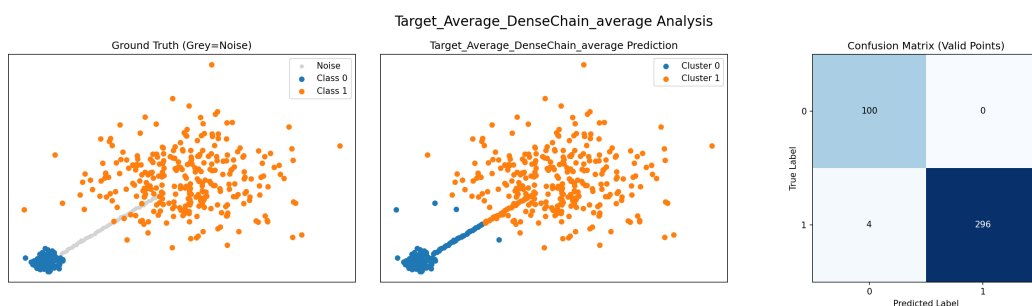


图 14: average-linkage 算法在高密度链条数据集上的聚类结果

实验结果显示该算法取得了近乎完美的聚类效果：

- 准确率 (Accuracy): 0.9900
- ARI: 0.9579

分析：average-linkage 算法在该挑战性数据集中展现了极强的鲁棒性。混淆矩阵显示，类别 0 的 100 个样本全部正确归类，类别 1 的 300 个样本中有 296 个正确归类，仅有极少数误判（主要是链条与大簇连接处的模糊点）。尽管存在高密度的噪声链条，但 average-linkage 在计算簇间距离时，考虑的是所有点对的平均距离。链条上的 60 个点相对于两个主簇（共 400 个点）而言，数量较少。当算法考虑合并左下角簇和右上角簇时，虽然链条部分距离很近，但两簇主体部分相距较远，拉高了整体的平均距离。这使得算法能够“忽略”局部的高密度连接，正确识别出两个主要的密度中心。同时，它对簇的直径限制不如 complete-linkage 那样苛刻，因此能够完整地保留右上角的稀疏大簇。

4. 高密度链条数据集算法性能总结

表4汇总了三种算法在高密度链条数据集上的性能。

表 4: 三种层次聚类算法在高密度链条数据集上的性能对比

算法名称	准确率 (Accuracy)	ARI
single-linkage 层次聚类算法	0.7475	-0.0033
complete-linkage 层次聚类算法	0.5050	-0.0654
average-linkage 层次聚类算法	0.9900	0.9579

该实验有力证明了 average-linkage 算法在应对**混合密度** (Varied Density) 和**结构性噪声** (如链条) 时的独特优势。它既避免了 single-linkage 的短视 (只看最近邻), 也避免了 complete-linkage 的僵化 (过度限制直径), 是一种在复杂真实场景中更为稳健的选择。

(五) 三种算法对比

基于在 UCI Wine 真实数据集以及两组特定合成数据集上的测试结果，结合算法的数学原理，我们对 Single-linkage、Complete-linkage 和 Average-linkage 三种层次聚类算法进行了系统的对比总结。

1. 算法特性与实验表现综合分析

1. Single-linkage (单链接):

- **核心优势：**对簇的几何形状没有假设，能够发现任意形状的簇，特别是嵌套、卷曲等非凸流形结构。本实验中，它是唯一能够完美识别 **同心圆数据集** 的算法。
- **主要劣势：**极度敏感于噪声，容易产生“链式效应”。只要存在少量的噪声“桥梁”，就会将本应分离的簇错误合并。这一缺陷导致其在 **UCI Wine 数据集** 和 **高密度链条数据集** 上均表现不佳。

2. Complete-linkage (全链接):

- **核心优势：**偏好紧凑、直径近似相等的球状簇，对噪声和离群点具有较好的抗干扰能力。在本实验中，它在 **UCI Wine 数据集** 上取得了最高的准确率，证明了其在处理类内紧凑、边界模糊的真实数据时的优越性。
- **主要劣势：**无法处理非凸形状（如同心圆），且对簇的大小（直径）和密度变化敏感。当簇的直径差异巨大时（如在 **高密度链条数据集** 中），它倾向于打碎大簇以减小最大直径，导致聚类失败。

3. Average-linkage (平均链接):

- **核心优势:** 作为一种折中方案, 它利用了簇内所有点的信息, 对测量噪声和局部密度变化具有极强的鲁棒性。在本实验的 **高密度链条数据集** 中, 只有它成功抵御了高密度噪声链的干扰, 正确分离了不同密度的簇。
- **主要劣势:** 对簇的形状仍有一定假设 (偏向凸形), 无法处理同心圆等复杂结构。此外, 在某些特定分布下 (如本实验中降维后的 Wine 数据), 如果类间平均距离不够显著, 其表现可能退化, 不如 Complete-linkage 稳健。

2. 综合性能对比表

为了更直观地展示三种算法的适用范围, 表5 对其进行了汇总对比。

表 5: 三种层次聚类算法的综合性能对比

算法	优点	缺点	本实验最佳场景
Single-linkage	能识别非凸、不规则形状; 仅需局部连接信息。	严重的链式效应; 对噪声极度敏感; 簇整体紧凑性差。	同心圆数据集 (非凸结构)
Complete-linkage	生成紧凑、直径近似的球状簇; 对链式效应免疫。	对离群点敏感 (若离群点拉大直径); 无法处理非凸或大小差异大的簇。	UCI Wine 数据集 (团状分布)
Average-linkage	对噪声和密度差异鲁棒性最强; 综合利用全局信息。	偏向发现凸形簇; 计算量相对较大 (需计算所有点对)。	高密度链条数据集 (混合密度与噪声)

综上所述, 在实际应用中, 如果数据呈现明显的流形结构 (如环状), 应优先选择 Single-linkage; 如果数据呈现团状且希望簇结构紧凑, Complete-linkage 是最佳选择; 而当数据中存在复杂的噪声背景或簇间密度差异较大时, Average-linkage 往往能提供最稳健的结果。

七、 实验总结与讨论

本实验基于 Python 与 NumPy 库自主构建了凝聚型层次聚类框架, 通过对比 single-linkage、complete-linkage 及 average-linkage 三种核心算法在 UCI Wine 真实数据集与特定合成数据集上的表现, 深入探究了不同距离度量方式对聚类性能的决定性影响。

1. 算法特性的验证:

- **Single-linkage:** 实验证实该算法仅依赖局部最近邻信息, 在处理 **同心圆数据集** 等 **非凸形状** 或 **流形结构** 时具有绝对优势 (准确率达 1.0000)。然而, 在 **UCI Wine 数据集** 与 **高密度链条数据集** 的测试中, 该算法极易受噪声点牵引, 产生严重的 “链式效应”, 将本应分离的簇错误合并。
- **Complete-linkage:** 该算法通过限制 **簇的直径**, 倾向于发现 **结构紧凑、直径近似相等** 的球状簇。在 **UCI Wine 数据集** 中, 它成功克服了链式效应, 表现 **优异** (准确率 0.8371), 证明了其在处理 **团状分布** 且 **边界模糊** 数据时的有效性。但实验也暴露了其在面对 **簇大小** 和 **密度差异** 剧烈的数据时 (如高密度链条) 的局限性。

- **Average-linkage:** 作为一种折中方案,该算法综合利用了 簇的全局结构信息。在 高密度链条数据集中,它展现了极强的 鲁棒性 (准确率 0.9900),成功抑制了 局部噪声干扰并适应了巨大的 密度差异,避免了 single-linkage 的短视与 complete-linkage 的僵化。
2. 适用边界的讨论: 实验结果表明,没有一种万能的层次聚类算法。数据分布的 几何拓扑特征直接决定了算法的选择:
- 当数据呈现嵌套、卷曲等 非凸流形特征时,应优先选择 **single-linkage**;
 - 当数据呈现 紧凑团状且希望聚类结果保持球形时, **complete-linkage** 是最佳选择;
 - 当数据中存在复杂的 结构性噪声或簇间 密度差异较大时, **average-linkage** 往往能提供更为稳健的聚类边界。

综上所述,本实验不仅实现了层次聚类的底层逻辑,更通过多维度的对比分析,清晰地界定了三种算法的优缺点与适用场景,达到了预期的实验目标。

八、 心得体会

通过本次实验,我从底层代码实现到高层数据分析,对层次聚类算法有了深刻的理解。

在**算法实现**层面,我体会到了不同距离度量方式对聚类行为的决定性影响。虽然三种算法的代码逻辑仅在距离计算上存在细微差别 (Min/Max/Avg),但其在特定数据集上的表现却截然不同。这让我认识到,没有万能的算法,只有最适合特定**几何拓扑特征**的算法。

在**数据验证**环节,通过设计“同心圆”和“高密度链条”等极端数据集,我直观地观察到了 **single-linkage** 的“链式效应”是如何发生的,以及 **average-linkage** 如何利用**全局结构信息**来抑制局部噪声干扰。这种“通过构造数据来攻击算法弱点”的测试方法,极大地锻炼了我的批判性思维。

在分析 **UCI Wine** 数据集时,我发现真实数据往往不具备完美的几何形态,且伴随大量噪声。在这种场景下, **complete-linkage** 对簇紧凑性的强制约束显得尤为重要,它能有效剥离噪声,发现数据的核心结构。

综上所述,本次实验不仅提升了我的编程能力,更让我学会了如何根据数据的**密度、形状和噪声分布**来选择最合适的聚类策略。

项目的完整实验报告与源代码已同步至 GitHub,详情请见:<https://github.com/fan-tuaner614/Machine-Learning>